

Exercícios – Objetivo 103.4

1. Compile o programa `prog.c` usando o comando

```
$ gcc -Wall -o prog prog.c
```

A seguir, partindo do comando acima, monte uma única linha de comando para (i) compilar o programa, (ii) armazenar a saída do GCC no arquivo `gcc.log` e (iii) mostrar quantos avisos (*warnings*) são exibidos durante a compilação.

2. Suponha que, no pipeline

```
$ cmd1 | cmd2 | cmd3 | cmd4
```

you queira usar o erro padrão do comando `cmd2` como entrada de `cmd3` (`cmd2` não produz nada na saída padrão). Como você poderia fazer isso sem usar arquivos intermediários?

3. Suponha que, no pipeline

```
$ cmd1 | cmd2 | cmd3 | cmd4
```

you queira usar **somente** o erro padrão do comando `cmd2` como entrada de `cmd3`, descartando a saída padrão de `cmd2` (ela não lhe interessa). Como você poderia fazer isso sem usar arquivos intermediários?

4. Considere a sequência de comandos abaixo:¹

```
$ mkdir vazio
$ cd vazio
$ cp a b
cp: cannot stat 'a': No such file or directory
$ cp a b >a
```

Por que não é mostrada mensagem de erro após o segundo comando `cp`? Qual o conteúdo do arquivo `a`?

5. Considere a sequência de comandos abaixo:

```
$ echo baiacu >a
$ cat a
baiacu

$ cp a b
$ cat b
baiacu

$ cp a b >a
$ cat b
```

Por que o arquivo `b` está vazio? O que há no arquivo `a`?

6. O que acontece se você usar uma variável não definida como *here string*? Existe alguma sinalização de erro? O comando recebe algum conteúdo?
7. Usando `find` e `xargs`, monte uma linha de comando para obter os hashes SHA-256 de todos os arquivos com extensão `.py` em um diretório.

¹Os exercícios 4 e 5 foram adaptados de http://wiki.inf.ufpr.br/maziero/doku.php?id=unix:shell_avancado

8. Crie alguns programas C no diretório corrente, usando arquivos com a extensão `.c` (você pode criar várias cópias de um mesmo programa, se quiser). A seguir, execute os dois comandos abaixo:

```
$ find . -name \*.c -exec grep -c \#include {} \;  
$ find . -name \*.c | xargs grep -c \#include
```

- (a) Como você explica as diferenças nas saídas dos dois comandos?
 - (b) Como você pode adaptar o segundo comando para gerar uma saída idêntica à do primeiro?
9. Os módulos do kernel do Linux são arquivos que possuem a extensão `.ko` e que residem no diretório `/lib/modules/versao`, onde *versao* é a versão do kernel. Usando `find` e `uname`, monte uma linha de comando para contar quantos módulos para a versão corrente do kernel existem. O comando deve funcionar sem alteração mesmo que o kernel instalado na máquina seja atualizado.